Assessment 2: Report

Analysis of Various Deep Learning Architectures for Animal Image Classification

Lim Dong Xian (14586049)

Richo Winson Tandoto (14558830)

Choo Pwint Chal (14733187)

Sin (14753575)

Aye Chan Moe (14644144)

Bachelor's Program, James Cook University

CP3501: Deep Learning

Mr. Randy Zhu

12 August 2025

Table of content

Introduction

In this report, we analyse an image classification dataset featuring seven animal classes, where we develop, train, and evaluate various deep learning models to achieve optimal performance through data preprocessing, data augmentation, and hyperparameter tuning. The project investigates how advanced techniques such as MixUp and Test Time Augmentation (TTA) can influence model accuracy and generalization. Detailed experimentation and analysis are presented below, highlighting how architectural choices and training strategies impact overall performance outcomes.

Competition dataset files: https://www.kaggle.com/datasets/limdx006/competition-files

Task 1

Kaggle Notebook-1 link: https://www.kaggle.com/code/limdx006/cp3501-2025-task-1-group-20

task-1-model link: https://www.kaggle.com/datasets/limdx006/task-1-model

Data Loading and Preprocessing:

We loaded the training features and labels CSV files, merged them on the image ID, and converted the multi-label one-hot encoded columns into a single categorical label column. We also constructed full image paths for easier access.

Data Exploration:

We analyzed the dataset by displaying the class distribution, previewing sample images for each class, and calculating basic image size statistics.

Data Preparation for Training:

Using FastAI's DataBlock API, we created a data pipeline that loads images and their labels, applies resizing, and splits the data into training and validation sets.

Model Training:

We trained a baseline image classification model using a ResNet34 architecture with FastAI's vision_learner, fine-tuning it for 5 epochs while monitoring accuracy.

Model Export:

Finally, we exported the trained model so it can be reused in the next tasks to do predictions.

Task 2

Kaggle Notebook-2 link: https://www.kaggle.com/code/limdx006/cp3501-2025-task-2-group-20

Load Model:

We loaded the trained model from Task 1 to use for predictions.

Data Preparation:

We loaded the training and test datasets, merged train features and labels, and created a new label column from one-hot encoded classes. Then, we rebuilt the data loader with a proper validation split.

Model Fine-tuning:

We unfroze the model layers and fine-tuned it with a low learning rate to improve accuracy and reduce validation loss.

Evaluation:

We calculated the log loss on the validation set to measure model performance.

Prediction & Submission:

We generated predictions on the test data, formatted the results to match competition requirements, and saved the submission file.

Reflection:

Our validation log loss was good, but the test log loss was much higher, indicating possible overfitting or dataset differences, and showing room for improvement.

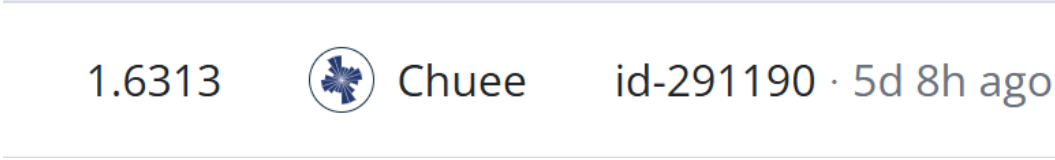
Task 3

For Task 3, we decided to test out 10 different model architectures, varying from resnet34 to efficientnet_b3. We did it by copying and editing task 2 code, only changing the model to make each prediction.

Model architectures:

1. beit_base_patch16_224

   Beit is a transformer model for images. It cuts an image into small patches (16 x 16) and learns how those fit together to understand the whole picture. The model itself has been trained on a huge amount of image data, so it can recognise patterns very well. Because transformers are good at finding both small details and broad patterns, they can deliver strong performance in image classifications.

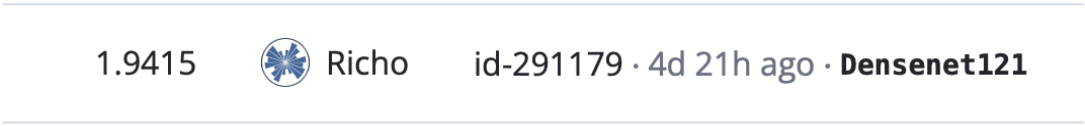


2. resnet101

   ResNet101 is a deep convolutional neural network with 101 layers, known for its use of residual connections (skip connections) that help the model train effectively even when very deep. It's widely used for image classification tasks because it can learn complex patterns without suffering from the vanishing gradient problem.

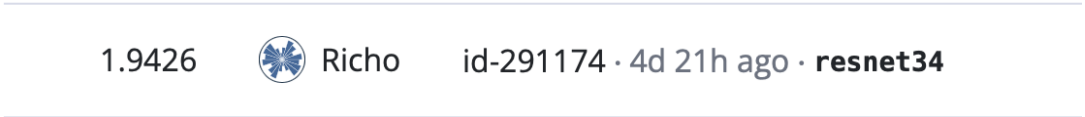
3. densenet121

   DenseNet121 is a type of CNN with 121 layers that uses dense connections by receiving inputs from the previous layers. It concatenates the output for better feature reuse with higher parameter efficiency while having an excellent gradient flow, as all layers are connected.



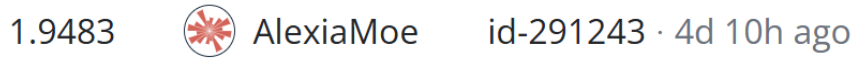
4. ResNet34

   ResNet34 is a type of CNN (Convolutional Neural Network) architecture that consists of 34 layers of neural network, where it makes use of the residual connections to avoid problems such as vanishing gradients. These connections allow the network to learn more effectively by skipping some layers, making training deeper models more stable. It has a medium size and medium speed compared to other models and is commonly used for image classification.



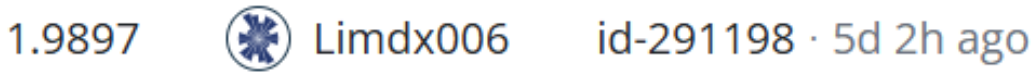
5. resnet50

   ResNet50 is a 50-layer deep convolutional neural network that became popular due to the incorporation of residual connections, facilitating the passage of a gradient through the network. By resolving the vanishing gradient issue, these skip connections improve the accuracy and stability of training deeper networks. It has a good trade-off between effectiveness and efficiency, making it popular in the use case of the classification of images.



6. Dino

DINO (Self-Distillation with No Labels) is a self-supervised learning method where a student model learns to match a teacher model's output on different views of the same image, enabling powerful visual feature learning without labelled data.

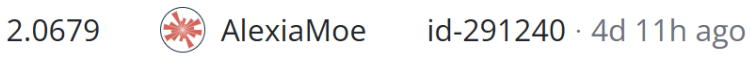


7. mobilenetv3_large_100

MobileNetV3 is a lightweight CNN model optimised for efficiency, achieving a balance between speed and accuracy. It incorporates depth-wise separable convolutions, which break down the processing into smaller, simpler steps to save time and computing power. It also has special activation functions to process images efficiently. Because it is both efficient and reliable, it was a good choice for testing and experimentation.

8. efficientnet_b0

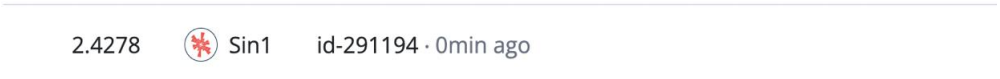
EfficientNet B0 is a small, light and scalable CNN architecture where



accuracy and efficiency are optimised based on a uniform scaling of depth, width and resolution with a compound coefficient. It employs such measures as mobile inverted bottleneck convolution and squeeze-excitation blocks that allow for low-cost but still high-performance. It is rather efficient and thus can find its application in real-time or under resource-limited conditions.

9. ConvNeXt Tiny

ConvNeXt Tiny is a lightweight CNN that blends transformer-inspired improvements with traditional convolutional design. It uses modern techniques like



large kernels and LayerNorm to achieve strong performance with high efficiency.

10. Efficientnet_b3

EfficientNet B3 is a compact and accurate CNN that scales depth, width, and resolution in a balanced way. It uses efficient MBConv blocks to deliver high

accuracy with fewer parameters and lower computational cost than older models.

Final Table:

Model name	Train_loss	Valid_loss	Accuracy	Validation Log Loss	Time
beit_base_patch16_224	1.530058	0.988503	0.652703	0.98850	04:45
ResNet101	0.873460	1.528533	0.417568	1.52853	02:06
DenseNet121	1.795173	1.256348	0.557658	1.25635	01:27
ResNet34	1.654726	1.194071	0.602703	1.19407	00:46
ResNet50	0.252983	0.206552	0.925225	0.20655	01:20
Dino	1.606716	1.835937	0.310811	1.83594	01:31
mobilenetv3_large_100	1.973910	1.397620	0.522973	1.39762	00:54
efficientnet_b0	0.247764	0.203510	0.926126	0.20351	01:22
ConvNeXt Tiny	0.426759	0.451028	0.842342	0.20582	05:37
Efficientnet_b3	0.883432	0.756522	0.732883	0.20887	01:42

Based on the results, we decided to select the best 5 performing models based on the submission score, which are beit_base_patch16_224, resnet101, densenet121, resnet34, and resnet50, to continue experimenting with data augmentations and hyperparameters.

Task 4

Types of Hyperparameters used

There are various types of Hyperparameters that we decided were significant enough to change the result in a meaningful way.

Epochs (default of 5)

- The amount of time a model went through the dataset, with the best value commonly in between underfitting and overfitting, depending on the dataset.

Random horizontal (default of True) and vertical flip (default to False)

- Animals can appear in different orientations in camera trap images.
- Flipping teaches the model not to depend on a fixed direction (like always expecting an animal to face right), which improves robustness.

Rotation (default of 10)

- Some cameras might not be perfectly level, or animals may be captured at odd angles.
- Adding rotation makes the model less sensitive to small orientation changes and improves spatial awareness.

Max_Zoom (default of 1.1)

- Animals may appear closer or farther from the camera.
- Zoom augmentation mimics this variability and helps the model learn features at different scales.

Lighting changes (default of 0.2)
- Lighting conditions can vary drastically in wildlife photography (e.g. day vs night, sun vs shadow).
- Adjusting brightness/contrast forces the model to focus on shape and texture, not lighting alone.

RandomErasing
- Helps the model learn to recognise animals even when parts of them are missing or obscured.
    - p=0.7: 70% chance to apply it
    - max_count=8: up to 8 areas can be erased
    - sh=0.1: each erased area is up to 10% of the image size

MixUp (default to False):
- Blends two images and their labels to regularise the model and prevent overfitting.

Warp (default of 0.2) :
- Geometrically disorient the shape of the image by stretching part of the image unevenly, making it more robust and applicable to the real world.

LabelSmooth (default of False):
- This function tells your model to use a special loss function called Label Smoothing Cross Entropy with a smoothing factor of 0.1.
- Instead of training the model on strict "100% correct" labels, it slightly softens the labels (like giving 90% confidence to the right answer and 10% spread over others). This helps the model avoid being too confident and improves its ability to generalise (reduce overfitting).

TTA (Test Time Augmentation):

- TTA (Test Time Augmentation) predicts multiple augmented versions of each image and averages the results for better accuracy.

Models chosen

Resnet101:

Notebook link (Aug and fine_tune without learning rate):
https://www.kaggle.com/code/limdx006/cp3501-2025-task-4-group-20-lim-resnet101

Notebook link (fine-tune with learning rate):
https://www.kaggle.com/code/limdx006/cp3501-2025-task-4-group-20-lim-resnet101-lr

Initial result = 1.7160

1. Training method (manual cycle with learning rate):
   - This training strategy first trains the model's head for 3 epochs with a learning rate of **3e-3**, then unfreezes all layers and fine-tunes the entire model for 5 more epochs with a gradually increasing learning rate from **1e-6 to 3e-5**.
   - This approach helps the model first learn general features safely, then gradually fine-tune deeper layers without large updates that could harm pretrained weights, improving overall performance.
2. Added augmentation like random flip, rotation, zoom, lighting, erasing, and mix up.

```
    batch_tfms = [
        *aug_transforms(mult=1.0, do_flip=True, flip_vert=True, max_rotate=15.0, max_zoom=1.2, ma
x_lighting=0.2),
        RandomErasing(p=0.7, max_count=8, sh=0.1)
    ]
```

Result:

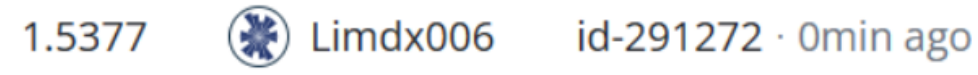
Initially, the ResNet101 model achieved a log loss of 1.7160. To improve generalisation and performance, data augmentation techniques were introduced — including random flips, rotation, zoom, lighting adjustments, and mix-up. This slightly reduced the log loss to 1.6809, suggesting minor improvements in the model's performance.

3. Remove redundant augmentation and use fine_tune instead of manual one-cycle training:

   learn.fine_tune(5)

   Result:

   The model was retrained using fine_tune without MixUp, which led to a more significant improvement, bringing the log loss down to 1.5377. This suggests that the previous setup may have suffered from over-regularisation or instability introduced by MixUp. Removing MixUp and allowing all layers to be fine-tuned helped the model better capture the underlying patterns, leading to better-calibrated predictions.



4. Try to increase the epoch to 10
   learn.fine_tune(10):

epoch	train_loss	valid_loss	accuracy	error_rate	time
0	1.357064	1.631643	0.420270	0.579730	02:06
1	1.106063	1.616132	0.415766	0.584234	02:06
2	0.893819	1.660207	0.457207	0.542793	02:06
3	0.757713	1.479827	0.483333	0.516667	02:06
4	0.658191	1.534300	0.501802	0.498198	02:06
5	0.557105	1.510564	0.505856	0.494144	02:06
6	0.493428	1.687114	0.492342	0.507658	02:06
7	0.451618	1.643164	0.505856	0.494144	02:06
8	0.396477	1.671398	0.506757	0.493243	02:06
9	0.383527	1.708513	0.504505	0.495495	02:06

Result:

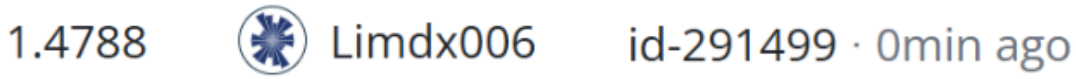
Training with 10 epochs seems to lead the model to overfitting, resulting in the model not performing well in unseen data.

5. 6 epoch and added Label Smooth
   - To prevent overfitting in future testing, I tried to introduce Label Smooth before training fine fine-tuning the model. And I decreased the number of epochs to 6.
   - Based on the training cycle on the 6th epoch compared to the 10-epoch training result, the gap between the train and valid loss is closer, which may also indicate that the model is generalising well, and a lower possibility leads to overfitting.

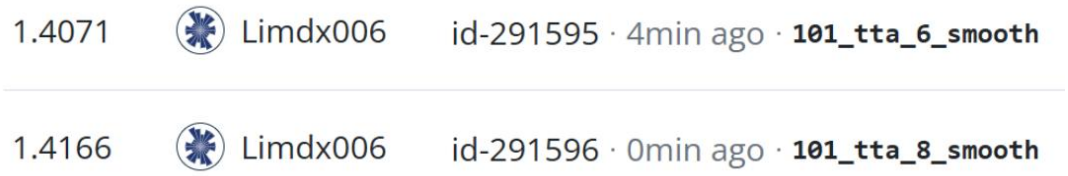
epoch	train_loss	valid_loss	accuracy	error_rate	time
0	1.549540	1.707100	0.416667	0.583333	02:06
1	1.294125	1.631763	0.434234	0.565766	02:06
2	1.120478	1.664243	0.468018	0.531982	02:06
3	1.020900	1.550426	0.486937	0.513063	02:06
4	0.948457	1.547921	0.489640	0.510360	02:06
5	0.916240	1.553430	0.498649	0.501351	02:06

Result:

This custom loss helped smooth the model's confidence in predictions and reduced overconfidence, leading to better generalisation.



6. Added TTA prediction with 6 and 8 epochs
   - So, as required by the task, TTA (Test Time Augmentation) is added to the final prediction.
   - Tried both 6 and 8 epochs with TTA and Label Loss.

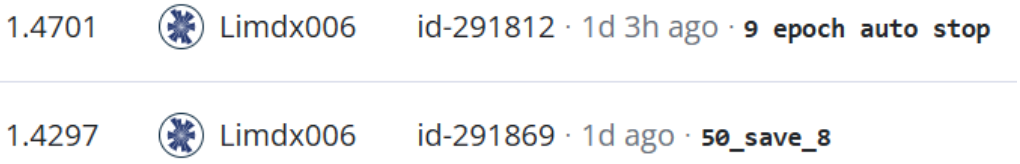


Result:

The result further improved after applying Test Time Augmentation (TTA) during prediction, which averaged predictions over augmented versions of test images, helping reduce variance and boosting robustness. I also tried training for 8 epochs, but performance did not improve further. This might indicate that 6 epochs was the optimal point for this setup currently.

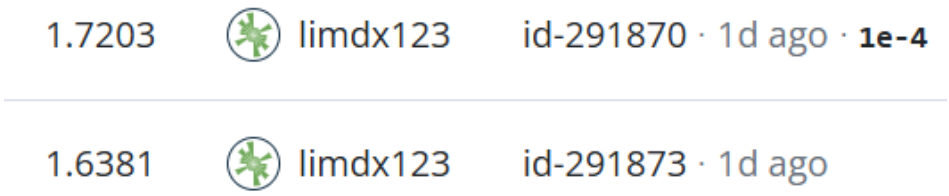
7. Other testing
   - To find out if this is the best approach, I also try other stuff. At first, I tried to play with the augmentation, like adding the rotation to 20 or more, but a performance drop.

   - Then I restore the augmentation to the best result I have and modify the training fine-tune to 50 and add an Early Stopping Callback when valid_loss didn't improve for 5 epochs. I try to run it twice and it stops on the 9th and 8th epoch, but the result is still not better than my 6 epoch.



   - To test out if Mix Up mess things up, I try it again with 6 epochs and augmentation. And the result is still getting worse.

- After that I try to train the model with different learning rate.
    - Recommended LR (valley zone): 1e-4
    - Moderately aggressive LR: 3e-4

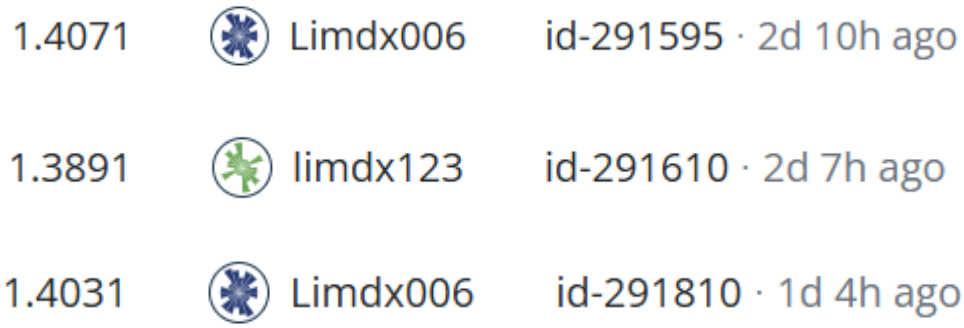


    - Conservative LR (safer generalisation): 3e-5

This might indicate that for ResNet101, manually setting the learning rate in fine-tune can give worse results because FastAI's default automatically finds and uses better learning rates for different layers.

8. Final result:
    - The final hyperparameter used will be 6 epochs without a learning rate.
    - The augmentation will enable random vertical flip, 15 rotation, 1.2 zoom, 0.2 lighting and erasing(70% probability, 8max count, 10% max patch size)

    I rerun the final setup 3 times as I want to add export code in it, and I submit the result to see if the result will drop for another try, but it comes out the result is quite stable around 0.41 ~ 0.38.



---

beit_base_patch16_224:

Notebook link (No Aug, Just TTA and fine_tune=5):

https://www.kaggle.com/code/choopwintchal/cp3501-2025-task-4-group-20-2nd-model-no-aug

Notebook link (Aug with TTA, MixUp):

https://www.kaggle.com/code/choopwintchal/cp3501-2025-task-4-group-20-2nd-model-mixup

Notebook link (Aug with TTA, learn_rate experiments)

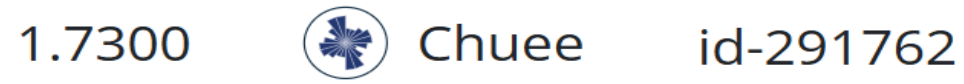
https://www.kaggle.com/code/choopwintchal/cp3501-2025-task-4-group-20-2nd-model/notebook

Initial result: from Task 3: 1.6313

Training method: StratifiedGroupKFold was used for splitting to ensure balanced labels and site diversity in validation.

1. No Augmentation No MixUp (only fine_tune = 5, TTA)
   For the first step, I kept the training setup the same as in Task 3 but used fine_tune (5, lr=3e-5) instead of the cycle and to TTA for inference. This change lowered the log loss from 0.9885 to 0,82948. The improvement is likely due to TTA averages predictions over several images reducing variance and making it more reliable.

2. Augmentation with TTA (learn_rate = 3e-5)
   Then I further added augmentations while having TTA to improve generalisation. The augmentations include horizontal flips, rotations (15 degree), zoom (1.5) and lighting adjustments (0.4). With these changes, the log loss increased to 1.28457.
   This drop could be because augmentations added a lot of variation to the image as well as the learning rate being small (3e-5) and only 5 epochs. With that change, it acts more like noise, making it harder for the model to fit the data and resulting in a higher log loss.



3. Augmentation with TTA, MixUp
   I tried adding MixUp with all the same settings as above. However, the log loss increased further to 1.34505. This means instead of improving robustness, the combination caused the model to underfit even more, leading to a higher log loss.

After the first three methods, I chose the best performing setup which is “Augmentation with TTA but no MixUp” and focused on changing the learning rate to see if the performance could be improved.

Learn-rate = 1e-4

This rate is often considered to be a safe midpoint allowing the model to adjust both shallow and deeper layers with 5 epochs while avoiding instability. The result is the log loss improved from (3e-5: 1.28457) to 0.96724, showing faster adaptation.

Learn-rate = 5e-4

Then to test whether stronger updates could accelerate convergence, I used 5e-4 with 5 epochs. The aggressive learn rate helped the model make better use of short training window, leading to a clear performance boost to 0.69809 log loss. It was underfitting at lower learning rates.

Learn-rate = 5e-4 with mixup

I further tested to see if combining the optimal learning rate with regularisation would yield better generalisation by using mixup. However, it increased label noise and may have overcompensated during the updates. Resulting in log loss increased to 0.80588 suggesting poorer performance.

Epoch = 50 with callback

Then, I tried extending the training to epoch 50 with callback after 5 consecutive no improvements. The validation loss stopped improving after 28 epochs, indicating overfitting. Although log loss (0.34532) was the lowest out of all the experiments, the score was 1.3921. It may be that the model began to memorise training data instead of learning features that generalize, which is why even though having excellent log loss, it failed to improve.

Epoch 10 with learn rate 5e-4

I experimented with training for 10 epoch with the learn rate 5e-4 to see if better improvements than 5 epochs. The result is log loss of 0.59605 which is better than 5 epoch's one. This seems to be the best balance between learn rate and generalization. It allowed faster adaptation to augmentation which 10 epochs gave enough time to converge without

overfitting. Moreover, disabling mixup helped avoid excessive regularisation and using TTA reduced prediction variance. Therefore, this setup produced the best overall performance across the experiments.

The final training setup includes:

- Horizontal flips
- Rotation up to 15 degrees
- Zoom 1.5x
- Max lighting 0.4
- Fine_tune with epoch = 10
- Learn rate = 5e-4

1.2476  Chooo

---

DenseNet121

Notebook link (Aug modified with TTA):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-aug-tta-group-20/notebook

Notebook link (MixUp):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-aug-tta-mixup-group-20

Notebook link (Added label smooth, 8 epochs):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-labelsmooth-8p-group-20/notebook

Notebook link (lr = 1e-3, vision_learner):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-lr-group-20

Notebook link (lr = 1e-3, EarlyStoppingCallback):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-lr-1-group-20

Notebook link (lr = 5e-4):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-lr-2-group-20

Notebook link (lr = 1e-3, 6 eporchs):

https://www.kaggle.com/code/sinnatherpaing/cp3501-2025-task-4-6p-group-20/edit

Initial result: 1.9415

1. Argumentations modified (with TTA)

   This experiment modified the augmentation pipeline, likely tweaking rotation, zoom, lighting, and other transforms, followed by applying Test Time Augmentation (TTA) during inference.
   TTA averages predictions over multiple augmented views of the test images, helping reduce variance and improve robustness.

   Result:
   The model achieved a log loss of **1.8530**, which was not a substantial improvement compared to previous baselines. This suggests that the augmentation changes may not have aligned optimally with the dataset, potentially introducing noise rather than beneficial variation.

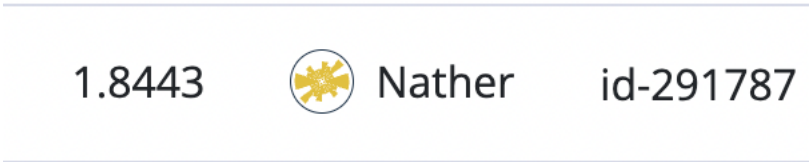
2. Mixup

   MixUp blends two images and their labels to create synthetic training samples, which can help regularisation and improve generalisation.

   Result:
   Log loss increased slightly to **1.8815**, indicating that MixUp in this setup may have been too aggressive or poorly matched to the dataset, possibly disrupting important spatial patterns in the images.

3. Added label smooth and try 8 epochs (No Mixup)

   Label smoothing reduces the model's overconfidence by softening hard target labels, which can improve generalisation. This test was run without MixUp and trained for 8 epochs.



   Result:
   The log loss improved to **1.8443** compared to MixUp, suggesting label smoothing

was more beneficial for this dataset. However, the improvement was still modest, and longer training did not drastically change the outcome.

4. Learning rates experiments

   Several learning rates and training strategies were tested:

   - Lr = 1e-3 (vision_learner): Achieved 1.4969 log loss — a notable improvement, suggesting that this rate allowed for efficient convergence without severe overfitting.
   - Lr = 1e-3 (EarlyStoppingCallback): Achieved 1.6954, worse than the vision_learner run, possibly due to premature stopping before optimal convergence.
   - Lr = 5e-4: Reached 1.5315, still better than most augmentation-heavy runs but slightly worse than the 1e-3 vision_learner.
   - Lr = 1e-3 with 6 epochs: Matched the vision_learner best score with 1.4956, confirming that this setup was optimal among the tested learning rate configurations.

   Learn-rate = 1e-3 (vision_learner)

   Learn-rate = 1e-3 (EarlyStoppingCallback)

   Learn-rate = 5e-4

   Learn-rate = 1e-3 with 6 epochs

5. Final result:

The best performance was achieved with learning rate = 1e-3 for 6 epochs, resulting in a 1.4956 log loss. For this dataset and model configuration, a simpler training pipeline with moderate augmentation, 6 epochs, and a learning rate of 1e-3 was the most effective approach, delivering stable and strong results without the drawbacks seen in more aggressive augmentation or overly long training.

ResNet34

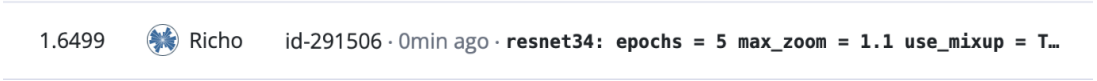
Notebook link: https://www.kaggle.com/code/richo7777/cp3501-2025-task-4-group-20

Initial result: 1.9426

1. Augmentations modified (step by step):
   a. MixUp (changed to True
      LabelSmooth (changed to True)
      TTA (changed to True)
   b. Max_zoom (changed from 1.1 to 1.3)
   c. Epoch (increase from 5 to 7)
      Activate EarlyStopCallingBack
   d. Flip_vertical (changed to True)
   e. MixUp (revert it back to False)

Training Method

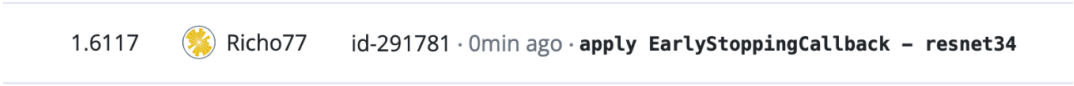
1. First, the code structure was slightly modified, with the only real change being the activation of MixUp, LabelSmooth and TTA which led to a significant improvement, going from a log loss of 1.9426 to 1.6499, which suggests that blending two images to regularize the model and reduces overfitting



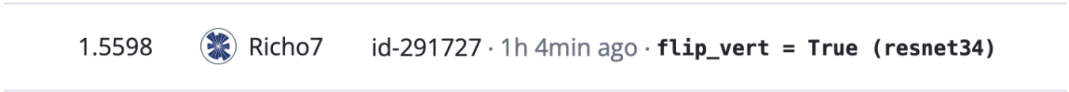
2. Then, I decided to change the max_zoom from 1.1 to 1.3 to check for improvements in model generalisation because increasing the zoom range allows the model to see more varied versions of the same image, potentially making it more robust to scale variations. It led to a decrease from 1.6499 to 1.5443, which suggests that there was a lot of noise that affected the images from before.

3. After that, I tried to apply EarlyStoppingCallBack, which runs the epoch infinitely until it reaches a point where the valid_loss does not improve for 2 epoch in a row, which is helpful to help find the limit for overfitting. Based on the result, it seems that it decreased in performance instead, so I decided not to include

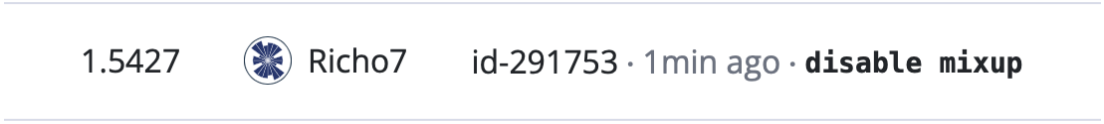
EarlyStoppingCallBack.



4. Then, I decided to activate flip_vertical to true, because logically it makes sense to test the model with different orientations of the image to fit real world applications. However, it seems that it is not the best result compared to when it was disabled, so I chose not to apply this change.



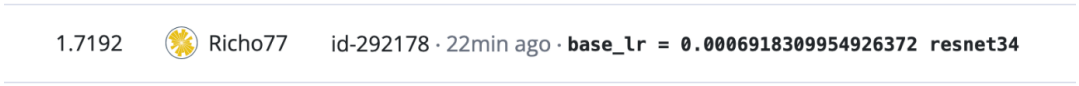
5. Finally, I removed MixUp while keeping LabelSmoothing active. This surprisingly results in a better log loss, which suggests that the combination of MixUp and LabelSmoothing caused the model to over regulate. Removing it allows the model to learn the patterns better because there is no unnecessary noise points anymore



Experiments with Learning rate:

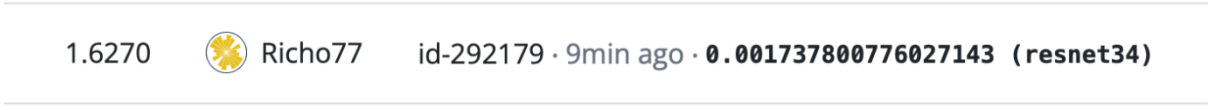
Lr = 0.0006918309954926372

Achieved a log loss of 1.7192, which was not ideal and worse from the original modified augmentation



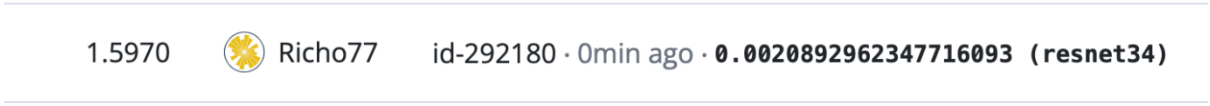
Lr = 0.001737800776027143

Achieved a log loss of 1.6270, could be due to the learning rate being too high which causes the model to not converge properly



Lr = 0.0020892962347716093

Achieved a log loss of 1.5970, although an improvement on previous attempts, still not better than the result without the implementation of learning rate.



In conclusion, the final augmentation used are:

- LabelSmooth (changed to True)
- TTA (changed to True)
- Epoch = 5
- Max_zoom = 1.3
- Everything else set to default

The improvements mainly came from Label Smoothing's ability to regularize predictions, TTA's benefit in using multiple augmented view, and the moderate zoom range providing useful variation without distorting key features. This final combination balances performance gains with training stability, making it a suitable configuration.

ResNet50

Notebook links:

TTA: https://www.kaggle.com/code/alexiawin/fork-of-fork-of-cp3501-2025-task-2-resnet50-tta

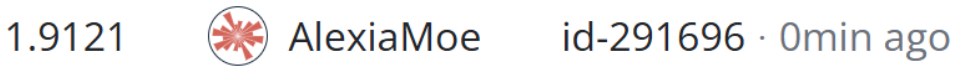
Augmentation: https://www.kaggle.com/code/alexiawin/fork-of-fork-of-cp3501-2025-task-2-resnet50augment

Label smoothing: https://www.kaggle.com/code/alexiawin/fork-of-fork-of-cp3501-2025-task-2-resnet50labelsm

Augmentation + Label Smoothing: https://www.kaggle.com/code/alexiawin/cp3501-2025-task3-resnet50augmentation-labelsmooth

Initial result: 1.9483

1. TTA

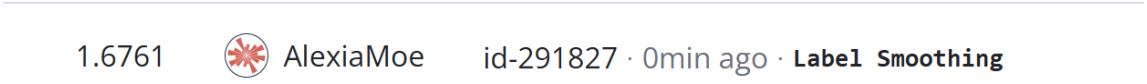


The TTA was applied when using the model in the inference stage, but there was no augmentation during training. Even with its ease, it scored a decent rank on the leaderboard. This is indicative of the fact that, without intricate training trickeries, TTA by itself can provide reliable performance by using numerous augmented views during testing.

2. Augmentation

I added training-time augmentation using aug_transforms() in the DataBlock after testing TTA, which produced a leaderboard score of 1.9121. I used max_rotate=10.0, max_zoom=1.1, max_lighting=0.2, max_warp=0.2, and flip_vert=True to enable vertical flipping. Such augmentations were there so that to enhance generalization by emulating real-model variation in the data. And yet, using these augmentations, the model scored a little lower 1.9149 though after training. This implies that practically augmentation is not an improvement since in this instance it performed no better than TTA and most likely because of the type of dataset used or the model already generalizing so well without augmentation.

3. Label smoothing alone



Label Smoothing was used to avoid overconfidence in prediction by the model and better generalize. An advantage of label smoothing over standard cross-entropy loss is that, instead of the model assuming total surety regarding the correct class, it lets the labels be predicted with a small degree of uncertainty. This has brought out substantial performance improvement over the previous methods. Label Smoothing had the best final leaderboard score of 1.6761 which was better than the TTA (1.9121) and augmentation alone (1.9149) strategies, and so it was the best method so far in enhancing model generalization.

4. Label smoothing + Augmentation

Having experimented with TTA, and then with augmentation, I chose to use data augmentation training but label smoothing as the loss. The augmentation had moderate changes, which included rotation, zoom, lighting, and warping, allowing vertical flipping so as to bring more variety to the images. The label smoothing prevented the overconfident predictions and assisted in regularizing the model, which is likely to enhance the generalization. The combination gave a score of 1.6262, in which all the previous methods I tested proved to be underperformers when used singularly. The enhancement reveals that label smoothing and augmentation complement each other in boosting model robustness and confidence in prediction.

Task 5

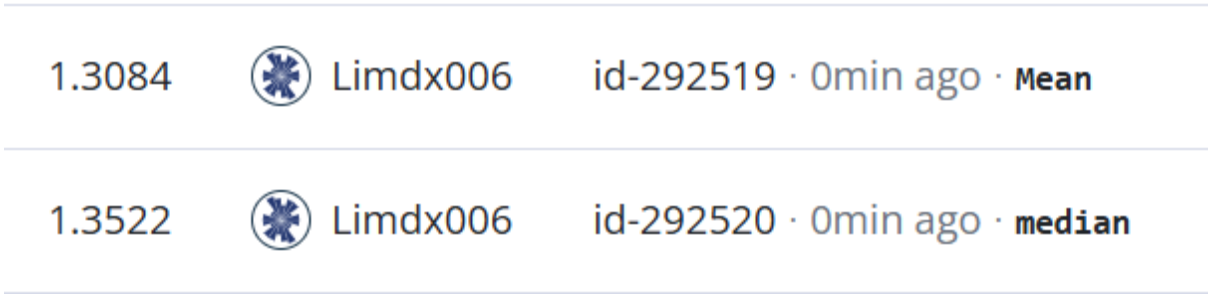
Notebook link: https://www.kaggle.com/code/limdx006/cp3501-2025-task-5-group-20-5-models

In Task 5, our group coordinated by having each member finalise their trained model and share the corresponding notebook containing the exported .pkl model files. Once everyone had imported their models, I began running the ensemble prediction pipeline by loading each model and generating predictions.

However, I encountered a significant issue where predictions from a single model were taking an unexpectedly long time—up to 2 hours per model. To diagnose the problem, I tried many ways to find and fix the issues. I checked the batch size, changed back to predict instead of TTA prediction, and recreated the individual data block for the test file, but none of these solutions worked. So, I checked on the device usage during prediction by inspecting the model's data loaders and processing device, only I discovered that all models imported are defaulted to running on the CPU, which caused the slow performance.

To fix this, I added (cpu=False) when importing the models to ensure they cannot use CPU and auto-switch to the Kaggle GPU environment. But, for the ResNet50 model, this automatic switch did not work as expected, so I manually moved the model to the GPU (cuda:0). After this adjustment, all models' prediction speed improved drastically.

Once the predictions were successfully generated, we adjusted the submission format to match the competition requirements, removing any extra columns such as unnecessary 'label' fields. The final submission scores were:



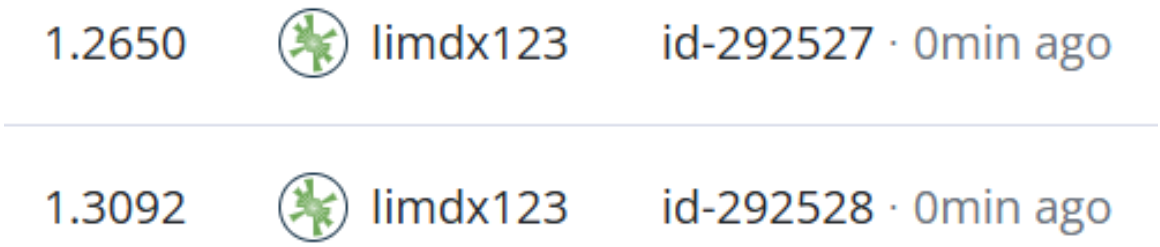
For comparison, the initial validation log loss for each model was:

- ResNet101: 1.4031
- ResNet34: 1.5970
- Densenet121: 1.4956
- ResNet50: 1.6262
- beit_base_patch16_224: 1.2476

This ensemble approach improved the log loss compared to most individual models, demonstrating the benefit of combining predictions to capture diverse feature representations and reduce overfitting. The mean ensemble outperformed the median, suggesting the averaging strategy was better at balancing outliers. While the ResNet224 model had the best

individual score, integrating all models helped stabilise predictions and improve overall robustness. This experience highlights the importance of hardware configuration and careful submission formatting in the model deployment process.

To verify my idea, I chose the best 3 models instead, which will be ResNet101, Densenet121 and beit_base_patch16_224.



Result: Mean (1.2650), Median (1.3092)

After selecting and averaging predictions from the three best-performing models, the results show a mean log loss of 1.2650 and a median log loss of 1.3092. While these results indicate an improvement compared to weaker models, the overall average score is still not better than the single best model. This is expected, as simple mean and median ensembling generally smooths predictions, reducing extreme errors, but it rarely surpasses the strongest individual model unless the combined models bring complementary strengths.

In all attempts, the mean score is better than the median score, which indicates that a few higher-loss samples (outliers) are pushing up the median, while the mean benefits from the ensemble's balancing effect. The mean being lower here is a positive sign, as log loss is the evaluation metric and a lower mean directly reflects better overall predictive performance.

Conclusion

In conclusion, our experiments across multiple architectures and training strategies showed that model performance in wildlife image classification is highly sensitive to the choice of hyperparameters, augmentations, and learning rates. While techniques like MixUp and heavy augmentation sometimes reduced accuracy. On the other hand, approaches such as moderate augmentation, label smoothing, and Test Time Augmentation generally improved generalisation. Optimal results were achieved by fine-tuning each model with carefully balanced settings, demonstrating that tailoring strategies to the model and dataset is key to achieving stable and competitive performance.

There is still space to improve the model performance by exploring more advanced ensembling techniques beyond simple averaging, as well as experimenting with additional data augmentation methods and learning rate schedules. Incorporating domain-specific knowledge or leveraging larger pretrained models could further enhance accuracy. Moreover, optimising training time and hardware usage would allow for more extensive

experimentation. Overall, continued fine-tuning and innovation in both model design and training strategies will be essential to push performance even higher.

Group Member

- Lim Dong Xian (limdx006)
- Aye Chan Moe (alexiawin)
- Richo (Richo7777)
- Chu (choopwintchal)
- Sin (sinnatherpaing)

Video link (YouTube): https://youtu.be/8FaeEee1AHk